TP8 | Mantenimiento, respaldo y recuperación

UTN FRM · Base de Datos II · FoodStore
Alumna: Pagano Amanda · Comisión 4 · Profesor: Neira Sergio
Repositorio: https://github.com/ALucrecia-Pagano/Base_de_datos2
Fecha de los ensayos: 24/10/2026 (UTC-03:00)

Objetivo y entorno

Evaluar la recuperación de FoodStore en un laboratorio Windows con PostgreSQL 17.11. Se ensayaron dos bases:

- `foodstore_dev`: el esquema original del TP1, como adaptación local de la base `foodstore` de la consigna.
- `foodstore_tp3_carga`: la base con carga masiva utilizada desde el TP3 en optimización, reportes e índices (TP3 a TP5). Por su volumen es la más representativa de un uso real, y permite ver cómo crece el tiempo de recuperación.

El destino descartable fue `foodstore_restore_test`. Las bases originales no se modificaron.

Procedimiento y validaciones

Se generaron respaldos en formato custom con `pg_dump -Fc`. Para restaurar, se creó el destino desde `template0` y se ejecutó `pg_restore --exit-on-error --no-owner --no-privileges`. Cada respaldo registra su salida en `logs/` y el resultado (PASS o FAIL, tamaño, SHA256 y horarios) en `metadata.json`, dentro de la misma carpeta fechada. Cada origen se ensayó por separado, uno después del otro. Autoricé por separado cada borrado de `foodstore_restore_test`.

Antes del respaldo y después de la restauración se registraron estos conteos exactos. Fueron idénticos en el ensayo previo y en la validación final:

Tabla	Dev antes	Dev restaurado	Carga antes	Carga restaurada
categoria	2	2	2	2
cliente	3	3	20.003	20.003
producto	3	3	50.003	50.003
pedido	5	5	200.005	200.005
detalle_pedido	7	7	499.571	499.571

Los nueve indicadores de integridad dieron cero en el ensayo previo y en la validación final. Esos indicadores son:

- precios negativos (el ensayo previo controló además precio cero);
- stock negativo;
- cantidades no positivas;
- subtotales incorrectos;
- cuatro tipos de referencias huérfanas.

En el ensayo previo, las restauraciones conservaron también 6 tablas, 5 vistas y 57 restricciones. La validación compara conteos e indicadores, no fila por fila. La restauración no incluye roles ni permisos.

Objetivos y resultados

Respaldos utilizados:

- **Scripts finales:** dev iniciado a las 23:04:14,748 (46.877 bytes); carga iniciado a las 23:07:31,677 (7.321.779 bytes).
- **Ensayo previo:**
  - **Dev:** iniciado a las 00:32:38,094, con 46.877 bytes.
  - **Carga:** 7.321.779 bytes, con marca registrada a las 00:32:43,629, justo antes de iniciar el respaldo. Es el mismo respaldo que se reutilizó en la repetición.

Indicador	Meta	Dev	Carga
RTO, scripts finales (24/10, 23:05 y 23:08)	≤ 15 min	**2,5788433 s**	**9,1510594 s**
RTO, ensayo previo (24/10, 00:32 y 00:37)	≤ 15 min	2,9817064 s	8,1943358 s
Antigüedad, scripts finales	RPO ≤ 24 h	**100,3614621 s**	**49,9573628 s**
Antigüedad, ensayo previo	RPO ≤ 24 h	0,7817533 s	aprox. 272,6074609 s

En los scripts finales, la antigüedad incluye el tiempo de autorización entre el respaldo y la restauración.

El RTO se mide desde inmediatamente antes de crear el destino hasta que terminan las validaciones.

La antigüedad del respaldo indica qué tan fresco era, pero no mide datos perdidos: las bases originales no recibieron escrituras durante el ensayo. El RPO de diseño lo fija la frecuencia diaria del respaldo, es decir, hasta 24 horas.

En el ensayo previo de carga, la antigüedad es mayor porque la restauración se repitió con el mismo respaldo. Como el inicio exacto de ese respaldo no quedó registrado, se usó la marca tomada justo antes.

Las metas se fijaron el 23/10, antes de los ensayos, y los valores observados quedan dentro de ellas. Son dos mediciones por base, en un laboratorio local, sin programación periódica instalada.

Mantenimiento justificado

El diagnóstico registró cero consultas ajenas activas y cero deadlocks acumulados en ese momento. Autovacuum y autoanalyze ya habían actuado sobre las tablas grandes. En cambio, `categoria`, `usuario` y `mv_resumen_ventas_categoria_mes`, por su tamaño reducido, no tenían estadísticas en la copia restaurada. Por eso se aplicó `ANALYZE` solo a esas tres relaciones y solo en el destino:

- conservaron 2, 3 y 26 filas;
- sus columnas con estadísticas pasaron de 0 a 5, 9 y 6.

No se aplicó VACUUM ni VACUUM FULL: las tablas grandes no tenían tuplas muertas y la consigna excluye VACUUM FULL como mantenimiento periódico. Tampoco se hizo mantenimiento sobre las bases originales. `maintenance.sql` incluye ahora la revisión de `last_autovacuum` y `last_autoanalyze` antes y después del `ANALYZE`. En la ejecución final, `maintenance.sql` registró que autovacuum ya había analizado las cuatro tablas grandes, mientras que las tres relaciones chicas tenían 2, 3 y 26 modificaciones, por debajo del umbral de autoanalyze, y `last_analyze` en NULL. Tras el `ANALYZE`, `last_analyze` pasó a 23:09:38 en las tres, con los conteos preservados.

Incidente

En el primer intento con la base de carga se detectó una diferencia entre origen y restaurada. Los valores eran iguales: solo cambiaba el orden de las claves del JSON.

Se corrigió la comparación para que fuera por valor, y se verificaron nuevamente, en vivo, la fuente y la restaurada. Autoricé borrar solo ese destino y repetir la restauración. El RTO del ensayo previo es el de la repetición.

`restore_test.ps1` ya incorpora las dos lecciones: compara tabla por tabla, así que no depende del orden de las claves, y guarda el resultado aunque falle.

Uso de IA y control humano

Controles propios previos a los ensayos. Con `psql` desde PowerShell:

- ejecuté los `COUNT(*)` de las cinco tablas en ambas bases, y coinciden con las líneas base;
- inspeccioné la estructura de `producto` y `detalle_pedido` con `\d`, y así se identificó que la columna real es `precio_lista` y no `precio`;
- corrí las validaciones de precio, stock y cantidad, que dieron cero.

También generé un respaldo preparatorio de `foodstore_tp3_carga` con `pg_dump -Fc`: comprobé el código de salida 0 y lo verifiqué con `pg_restore --list`. Decidí que el respaldo se hiciera sobre la base original y no sobre una copia, por eso `foodstore_dev` es el origen principal.

Asistencia de IA. La SPEC inicial se redactó con asistencia de IA. Codex generó los scripts y ejecutó el ensayo previo. La cátedra admite el uso de agentes, y se utilizó Codex en lugar de OpenCode.

Cada operación sobre las bases requirió mi autorización: respaldos, borrado de cada destino, repetición y `ANALYZE`. Hay dos constancias en las evidencias:

- mi autorización textual ("si autorizo") en `repeticion/timing.json`;
- el caso del segundo borrado, registrado en `evidencias/LEEME.md`: cuando Codex intentó borrar el segundo destino apoyándose en la autorización del primero, el sistema de revisión de la herramienta lo bloqueó, y el borrado se hizo solo después de mi autorización específica.

Pruebas de los scripts finales.
El 24/10, entre las 23:04 y las 23:09, se respaldaron y restauraron ambas bases con los scripts entregados. Claude Code ejecutó cada operación después de mi autorización explícita:

- `pruebas/test_delivery.ps1`: 14 de 14 pruebas aprobadas con herramientas simuladas (salida guardada a las 23:19).
- `backup_foodstore.ps1`: PASS en ambas bases, con los nueve controles en cero y el dump nombrado con fecha y hora.
- `restore_test.ps1`: PASS en ambas restauraciones, con conteos idénticos a la línea base y los nueve controles en cero.
- Antes de cada borrado de `foodstore_restore_test` se comprobó que no tuviera conexiones.

Revisión posterior. No leí en detalle los cuatro scripts antes de su ejecución. Los revisé después con Claude Code, que también usé para auditar este informe y la SPEC y para contrastar los números con las evidencias. Los cambios los decidí yo. Para la ejecución final, revisé y aprobé cada cambio de código antes de aplicarlo y autoricé cada operación de forma individual. Los cambios de código de esta etapa (nueve controles, nombre del dump con fecha, cierre con ROLLBACK y revisión de autovacuum) los aplicó Claude Code después de mi aprobación.

Mejora y conclusión

Mejora para la próxima ejecución: respaldar también roles y permisos con `pg_dumpall --globals-only`, restaurarlos antes del dump y quitar `--no-owner --no-privileges` de la restauración. Hoy se recuperan los datos, pero no los dueños ni los permisos. Además, programar el respaldo diario con el Programador de tareas de Windows haría que el RPO de 24 horas no dependa de ejecutarlo a mano.

Conclusión: las dos bases se recuperaron con conteos idénticos e indicadores de integridad en cero, con tiempos muy por debajo de las metas, en un laboratorio local.

Evidencias:

- Carpeta del TP8 en el repositorio: https://github.com/ALucrecia-Pagano/Base_de_datos2/tree/main/TP8_Mantenimiento_Respaldo_Recuperacion
- `evidencias/results.json`;
- `evidencias/LEEME.md`;
- `evidencias/foodstore_tp3_carga/repeticion/timing.json`;
- `evidencias/salud_mantenimiento/`;
- `evidencias/validacion_final_20261024/`.

Los respaldos `.dump` se conservan fuera del repositorio.